AI 工具使用详情说明

一、 所用 AI 工具信息

工具名称	版本 / 型号	开发机构	使用日期
Doubao	Doubao-4	北京字节跳动网络技术有限公司	2026 年 4 月 17 日
DeepSeek	DeepSeek-R1-0528	深度求索	2026 年 4 月 17 日

二、 问题一

2.1 具体使用目的和应用环节

在“问题一”的数据预处理及模型建立与求解阶段，AI 工具主要用于代码辅助编写、算法逻辑梳理

1. 数据预处理环节：使用 Doubao 辅助编写了基于“孤立森林”进行无监督异常检测，以及基于“随机森林”进行多重插补重构的 Python 基础代码框架 。
2. 核心预警模型构建：使用 Doubao 辅助构建了代价敏感 LightGBM 分类底座，并生成了基于高斯过程的 Optuna 贝叶斯超参数寻优代码框架 。

2.2 关键交互记录

2.2.1 交互记录 1（针对多重插补与异常检测）

人工提示词：

“请帮我写一段 Python 代码：首先使用孤立森林算法检测数据中的异常值，再使用随机森林多重插补算法对检出的异常值进行处理”

AI 回复如下：

我为你编写了完整可直接运行的 Python 代码，实现了孤立森林（Isolation Forest）检测异常值 + 随机森林迭代多重插补处理异常值的全流程。代码包含详细注释，同时适配新手使用，你只需替换自己的数据集即可。

核心流程

1. 生成 / 导入数值型数据集
2. 孤立森林算法识别异常值（标记为 -1）
3. 将检测出的异常值替换为缺失值（np.nan）
4. 随机森林多重插补填充缺失值
5. 输出对比结果

完整代码

python

```python
# ==================== 1. 导入依赖库 ====================
import pandas as pd
import numpy as np
# 孤立森林（异常检测）
from sklearn.ensemble import IsolationForest
# 随机森林 → 迭代式多变量插补（处理缺失值）
from sklearn.ensemble import RandomForestRegressor
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

# ==================== 2. 固定随机种子（保证结果可复现） ====================
np.random.seed(42)

# ==================== 3. 构造/导入数据集 ====================
# 【方式一：使用模拟数据（仅写单位）】
# 生成300行3列的示例数据
data = pd.DataFrame({
    '特征1': np.random.normal(50, 10, 300),
    '特征2': np.random.normal(20, 5, 300),
    '特征3': np.random.normal(80, 15, 300)
})
# 手动插入3个极端异常值（模拟真实场景）
data.iloc[10:15, 0] = 200  # 特征1异常
data.iloc[20:25, 1] = 100  # 特征2异常
data.iloc[30:35, 2] = 10   # 特征3异常

# 【方式二：替换为你自己的数据】
# data = pd.read_csv("你的数据路径.csv")  # 读取CSV
# data = pd.read_excel("你的数据路径.xlsx")  # 读取Excel
# 注意：仅保留**数值型特征**，非数值列需先删除/编码

# ==================== 4. 孤立森林检测异常值 ====================
# 初始化孤立森林模型
# contamination: 异常值比例（auto表示自动判断，也可手动设0.05-0.2）
iso_forest = IsolationForest(n_estimators=100, contamination='auto', random_state=42)

# 训练模型并预测异常值
# 返回结果：1=正常样本，-1=异常样本
outlier_labels = iso_forest.fit_predict(data)

# 统计异常值数量
n_outliers = np.sum(outlier_labels == -1)
print(f"🔍 孤立森林检测完成，共发现 {n_outliers} 个异常值")

# ==================== 5. 将异常值替换为缺失值 ====================
# 创建异常值掩码（True=异常，False=正常）
outlier_mask = outlier_labels == -1
data_with_nan = data.copy()
# 把检测出的异常值替换为缺失值（用于后续插补）
data_with_nan[outlier_mask] = np.nan

# ==================== 6. 随机森林多重插补处理缺失值 ====================
# 初始化多重插补器，使用基于随机森林的回归
imputer = IterativeImputer(
    estimator=RandomForestRegressor(n_estimators=100, random_state=42),
    max_iter=10,  # 迭代次数
    random_state=42
)

# 执行插补
data_imputed = pd.DataFrame(
    imputer.fit_transform(data_with_nan),
    columns=data.columns
)

# ==================== 7. 结果预览 ====================
print("\n===== 原始数据（前5行） =====")
print(data.head())
print("\n===== 标记异常值为缺失后的数据（前5行） =====")
print(data_with_nan.head())
print("\n===== 随机森林多重插补后数据（前5行） =====")
print(data_imputed.head())

# ==================== 8. 保存处理后的数据（可选） ====================
# data_imputed.to_csv("已异常值处理数据.csv", index=False, encoding='utf-8-sig')
```

2.2.2 交互记录 2（针对代价敏感模型与贝叶斯优化）

人工提示词：

“依据我的医疗数据集，请使用 LightGBM 构建一个二分类预警模型，要求引入非对称代价矩阵赋予少数类更高权重，并结合贝叶斯超参数自适应寻优。”

AI 回复如下：

我为你定制了适配医疗数据、解决严重类别不平衡的完整解决方案：基于 LightGBM 构建二分类预警模型，嵌入非对称代价矩阵（赋予少数类 / 病患样本更高错分代价），结合 Optuna 贝叶斯优化 自动调参，全程贴合医疗场景的核心需求（降低漏诊率、优先保障召回率）。

核心亮点（医疗场景专属）

1. 非对称代价矩阵：严格区分「漏诊（少数类判错）」和「误诊（多数类判错）」的代价，漏诊权重远高于误诊；
2. 类别不平衡适配：分层采样、scale_pos_weight 加权、拒绝使用准确率作为评估指标；
3. 贝叶斯调参：Optuna 自动寻找最优超参，比网格搜索效率提升 10~100 倍；
4. 医疗专属评估：重点输出召回率（灵敏度）、AUC、F1 分数，而非无效的整体准确率；
5. 原生支持医疗数据：兼容类别特征、缺失值，无需繁琐独热编码。

2.3 采纳内容和人工修改情况

1. 采纳情况：采纳 AI 生成的模型构建基础框架。
2. 人工修改：AI 生成的代码仅为通用框架。结合具体场景，进行了大量人工修正：
3. 参数干预：在孤立森林检测中，修改异常值比例、训练参数、修改路径。
4. 逻辑重构： 结合我的模型建立，重新构建 LightGBM+贝叶斯优化部分代码，进行代码调参

问题二

2.4 具体使用目的和应用环节

在“问题二”的建模与求解中，AI 工具主要辅助完成了以下任务：

风险概率映射：使用 DeepSeek 辅助编写了广义加性模型（GAM）结合样条基函数的 Python 实现代码，用于将离散标签转化为连续生理风险概率。

2.5 关键交互记录

2.5.1 交互记录 1（关于 GAM 与样条函数）

人工提示词：

“我需要构建一个高血脂症预警模型。为了避免目标变量泄露，我剥离了理化指标，现在想用 GAM 模型结合样条函数将二分类标签平滑映射为连续概率，请给出 Python 实现方案。”

AI 回复如下：

以下是基于GAM（广义加性模型）和样条函数实现高血脂症二分类概率预警的Python完整方案。该方案**不使用任何理化指标**，完全依赖非理化特征（如年龄、BMI、饮食评分、运动频率等），避免目标变量泄露，并将离散标签平滑建模为连续概率。

---

1. 环境准备

需要安装 `pygam` 及其依赖：

```bash
pip install pygam scikit-learn matplotlib seaborn
```

---

2. 原理简述

GAM 的形式为：

$$g(\mathbb{E}[y|x]) = \beta_0 + \sum_{j=1}^{p} f_j(x_j)$$

其中 g 为链接函数（逻辑回归中为 **logit**），f_j 为特征 x_j 的平滑函数，这里采用**三次B样条**基展开并加上惩罚项防止过拟合。最终输出：

$$P(y=1|x) = \frac{1}{1+\exp\left(-\left(\beta_0 + \sum f_j(x_j)\right)\right)}$$

相比传统逻辑回归，GAM可以自动学习**非线性**与**交互作用**（通过特征间的加性组合），同时保持较高的可解释性，非常适合医疗预警场景。

---

3. 数据准备（示例）

假设你的数据集已经剥离了所有理化指标，仅包含以下特征：

特征名	类型	说明
age	连续	年龄（岁）
bmi	连续	身体质量指数
diet_score	连续	饮食健康评分（0-100）
exercise_freq	连续	每周运动次数
family_history	二分类	是否有家族史（0/1）
gender	二分类	性别（0男/1女）

python 复制 下载

```python
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import roc_auc_score, average_precision_score, brier_score_loss, classification
_report

# 生成模拟数据（请替换为你自己的数据）
np.random.seed(42)
n_samples = 2000

data = pd.DataFrame({
    'age': np.random.normal(55, 12, n_samples),
    'bmi': np.random.normal(26, 4, n_samples),
    'diet_score': np.random.uniform(30, 100, n_samples),
    'exercise_freq': np.random.poisson(2, n_samples),
    'family_history': np.random.binomial(1, 0.3, n_samples),
    'gender': np.random.binomial(1, 0.5, n_samples)
})

# 构造真实的概率（非线性关系）
log_odds = (
    -3.5
    + 0.03 * data['age']
    + 0.05 * (data['bmi'] - 25)**2 / 10
    - 0.02 * data['diet_score']
    - 0.2 * data['exercise_freq']
    + 0.8 * data['family_history']
    + 0.1 * data['gender']
    + 0.005 * data['age'] * data['bmi'] / 10   # 轻度交互
)
prob = 1 / (1 + np.exp(-log_odds))
data['label'] = np.random.binomial(1, prob)

# 划分训练集和测试集
X = data.drop('label', axis=1)
y = data['label']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

4. 使用 pyGAM 构建 Logistic GAM

`pygam` 中的 `LogisticGAM` 支持二分类，默认对每个连续特征使用 **样条基**，对分类特征可用 `Term` 或

2.6 采纳内容和人工修改情况

采纳情况：
采纳了 AI 提供的 GAM 模型框架的标准调用代码。
人工修改：
依据真实情况，修改数学公式

$$g(p_i) = \ln\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \sum_{j=1}^{p} f_j(x_{ij}) \tag{21}$$

2.7 问题三

无，未使用 AI